Pipes in the Desert
Skeleton Program

E5 - TEAM A

Supervisor:

Dr. Balla Katalin

Members:

Muhammad Ibrahim Shoeb	OZLVV3	muhammadibrahimshoeb@gmail.com
Arda Gecegörür	EKQNAK	ardagecegorur@gmail.com
Ilgın Tunç	TDQWFF	ilgintunc11@gmail.com
Muhammad Hameez Khan	TFBB32	hameezkhan993@gmail.com
Yahya Akhrikhar	K20UXP	yahyaakhrikhar02@gmail.com
Aasif Mohd	OI6VE6	aasifuk1122@gmail.com

March 28th, 2026

6. Skeleton program

6.1 Deployment guide

6.1.1 List of files

File Name	Size	Date	Content
System.java	13 KB	28.03.2026	Main game coordinator class; manages session lifecycle, turn order, timer, and end-game logic
Player.java	14 KB	28.03.2026	Abstract base class representing a game participant; handles movement and turn state
Plumber.java	20 KB	28.03.2026	Extends Player; implements plumber-specific actions: repair, connect, insert pump, pick up components
Saboteur.java	7 KB	28.03.2026	Extends Player; implements saboteur-specific actions: puncture pipe, change pump direction
PipeNetwork.java	15 KB	28.03.2026	Stores and manages the full network topology; handles adjacency, connection, and pump insertion
NetworkElement.java	5 KB	28.03.2026	Abstract base class for all physical network elements; provides shared connectivity behavior
Pipe.java	9 KB	28.03.2026	Represents a water-carrying segment; tracks puncture state, free end, occupancy, and water
Pump.java	11 KB	28.03.2026	Active routing element; manages input/output selection, breakdown state, and connection limit
Spring.java	4 KB	28.03.2026	Water source node; produces water each simulation step
Cistern.java	8 KB	28.03.2026	Water destination node; stores delivered water and generates new pipes and pumps

ScoreBoard.java	3 KB	28.03.2026	Tracks and compares delivered and leaked water; determines the winning team
GameTimer.java	3 KB	28.03.2026	Manages the countdown timer; triggers end-game when time expires
WaterFlowManager .java	5 KB	28.03.2026	Evaluates water movement each tick; registers delivered and leaked water
Team.java	6 KB	28.03.2026	Groups players of the same role; provides team-level information and member access
Main.java	34 KB	28.03.2026	Entry point of the program; displays the welcome message, menu, and launches the game session

6.1.2 Compilation

1. Prerequisites

Before compiling and running the project, ensure the following tools are available on your system.

1.1 Required Software

- Java Development Kit (JDK) — version 11 or higher (JDK 17 recommended)
- A terminal: Command Prompt or PowerShell (Windows) / Terminal (Linux, macOS)
- Maven — only required if using the Maven build method (optional)

1.2 Verify Java Installation

Open a terminal and run the following commands to confirm Java is properly installed:

```
java -version
javac -version
```

Both commands should report version 11 or higher. If either command is not recognised, install the JDK from https://adoptium.net before proceeding.

2. Extract the Project

Unpack the project archive using the method appropriate for your operating system.

2.1. Windows

Right-click the archive file → Extract All
Choose a destination folder and confirm

2.2. Linux / macOS

```
unzip Pipes-In-The-Desert.zip
```

3. Navigate to the Project Directory

Open a terminal and change into the extracted project folder:

```
cd path/to/Pipes-In-The-Desert
```

Replace path/to/Pipes-In-The-Desert with the actual path where the archive was extracted. All subsequent commands must be executed from this directory.

4. Compile the Project

The Java source files are located under src/main/java/. Choose one of the two build methods below.

4.1 Method A — Plain javac (Primary)

Create the output directory and compile all source files with the following commands:

Windows (Command Prompt):

```
mkdir out
javac -d out -sourcepath src/main/java
src/main/java/app/Main.java
```

Linux / macOS:

```
mkdir -p out
javac -d out -sourcepath src/main/java
src/main/java/app/Main.java
```

Compiled .class files will be placed in the out/ directory. No errors or warnings should appear on **success.**

4.2 Method B — Maven (Optional Alternative)

If Maven is installed and a pom.xml file is present in the project root, you may use:

```
mvn compile
```

Maven will automatically resolve dependencies, compile sources, and place output in the target/ directory.

5. Troubleshooting

If javac is not recognised, the JDK is not installed or not on PATH. Install JDK 11 or higher and add it to your system PATH.

If **`Main.java`** is not found, confirm you are in the project root directory and that the path **`src/main/java/app/Main.java`** exists.

If **`app.Main`** is not found at runtime, ensure compilation succeeded and that **`out/app/Main.class`** exists before running.

If the Maven build fails, verify Maven is installed by running mvn -version and that a valid pom.xml is present in the project root.

6.1.3 Run

After successful compilation, the program can be started from the project root directory with the following command:

```
java -cp out app.Main
```

When the program starts, the user is greeted by the main menu:
Welcome to Pipes in the Desert!

1. Start Game
2. Instructions
3. Exit

The user selects an option by entering the corresponding number from the keyboard.

1. Main Runtime Behavior

The skeleton program is controlled through console menus and interactive questions.
After selecting Start Game, the program:

- asks for the initial game data,
- initializes the game session,
- creates the initial network,
- starts the timer,
- and enters the running state.

During runtime, the system executes the Run Game loop, which includes:

- updating round events,
- generating new components when conditions are met,
- randomly breaking pumps when applicable,

- recalculating water flow,
- advancing turn order,
- and checking whether the game should end.

Player-controlled actions are selected through menus.

When a player is in a situation where a conditional action is possible, the program asks an interactive yes/no question.

Examples of such questions include:

- Do you want to puncture the pipe? (yes/no)
- Do you want to repair the pipe? (yes/no)
- Do you want to repair the pump? (yes/no)
- Do you want to pick up the generated pipe? (yes/no)
- Do you want to pick up the generated pump? (yes/no)

Test Scenarios

Test 1: Main Menu → Start Game

Sequence:

- Main.main()
- System.startGame()
- System.initializeGame()
- GameTimer.start()

Expected Results:

1. The system asks for:
   -player name:
   -choose team:
      1. Plumber
      2. Saboteur
   -choose game duration:
      1. 1.3 minutes
      2. 2.5 minutes
2. The main game objects are created.
3. The initial network is generated.
4. The timer starts and the game enters the running state.

TEST 2: Run Game Loop

Sequence:

- System.updateRoundEvents()
- WaterFlowManager.recalculateFlow()
- System.nextTurn()
- System.checkEndCondition()

Expected Results:

- The round state is updated.
- Water flow is recalculated.
- Turn control advances to the next player.
- The system checks whether the end condition is reached.

TEST 3: Move Player

Sequence:

- Player.moveTo(target)
- Player.setPosition(target)

Expected Results:

- The player selects a target pipe or pump.
- If the move is valid, the player position is updated.
- If the move is invalid, an error message is displayed:
  "Invalid move, try another!"

TEST 4: Saboteur → Puncture Pipe

Sequence:

- Saboteur.puncturePipe(pipe)
- Pipe.puncture()
- WaterFlowManager.recalculateFlow()
- WaterFlowManager.registerLeakedWater(amount)

Expected Results:

- If the saboteur is in a valid position, the system asks:
  "Puncture the pipe: (Yes,No)"
- If the answer is yes, the pipe becomes punctured.
- Water leakage is registered and displayed.

TEST 5: Plumber → Repair Pipe

Sequence:

- Plumber.repairPipe(pipe)
- Pipe.repair()
- WaterFlowManager.recalculateFlow()

Expected Results:

- If the plumber is on or adjacent to a punctured pipe, the system asks:
  "Repair the pipe:(Yes,No)"
- If the answer is yes, the pipe is repaired and flow is recalculated.

TEST 6: Plumber → Repair Pump

Sequence:

- Plumber.repairPump(pump)
- Pump.repair()
- WaterFlowManager.recalculateFlow()

Expected Results:

- If the plumber is on or adjacent to a broken pump, the system asks: "Repair pump: (Yes, No)"
- If the answer is yes, the pump is restored and flow is recalculated.

TEST 7: Change Pump Direction

Sequence:

- Plumber.changePumpDirection(...) / Saboteur.changePumpDirection(...)
- Pump.setDirection(input, output)
- WaterFlowManager.recalculateFlow()

Expected Results:

- The player selects a working pump.
- The system allows the player to choose new active input/output pipes.
- The pump direction is updated and water flow is recalculated.

TEST 8: Disconnect a Pipe → Connect a Pipe

Sequence:

- Pipe.disconnectEnd()
- PipeNetwork.disconnectElements(a, b)
- Plumber.connectPipeEnd(pipe, target)
- Pipe.connectFreeEnd(target)
- PipeNetwork.connectElements(a, b)
- WaterFlowManager.recalculateFlow()

Expected Results:

- The selected pipe end is disconnected.
- A free end is created.
- The plumber reconnects the free end to a valid adjacent target.
- The system updates the structure and recalculates water flow.

TEST 9: Pick Up Pump at Cistern

Sequence:

- Cistern.hasAvailablePump()
- Cistern.takePump()
- Plumber.collectPumpFromCistern(cistern)

Expected Results:

- If the plumber is adjacent to a cistern with an available pump, the system asks: "Pick up pump: (Yes, No)"
- If the answer is yes, the pump is collected successfully.

TEST 10: Pick Up Pipe from Cistern

Sequence:

- Cistern.hasAvailablePipe()
- Cistern.takePipe()

Expected Results:

- If the plumber is adjacent to a cistern with an available pipe, the system asks: "Pick up pipe: (Yes, No)"
- If the answer is yes, the pipe is collected successfully.

TEST 11: Place New Pipe

Sequence:

- placement validation
- network connection update
- WaterFlowManager.recalculateFlow()

Expected Results:

- If the plumber carries a previously collected pipe, the system allows the plumber to choose a placement location.
- The system checks whether the placement is valid.
- If valid, the pipe is inserted into the network and the updated network state is displayed.

TEST 12: Insert New Pump

Sequence:

- PipeNetwork.insertPump(targetPipe, pump)
- Plumber.insertPump(targetPipe, pump, network)
- WaterFlowManager.recalculateFlow()

Expected Results:

- If the plumber carries a pump, the system allows the plumber to choose a target pipe.
- The selected pipe is split and the new pump is inserted.
- The system recalculates water flow.

TEST 13: Generate New Pipe

Sequence:

- System.updateRoundEvents()
- System.requestComponentGeneration()
- Cistern.generatePipe()

Expected Results:

- When the pipe generation condition is satisfied, a new pipe is generated at the cistern.
- The pipe remains available until picked up by a plumber.

TEST 14: Generate New Pump

Sequence:

- System.updateRoundEvents()
- System.requestComponentGeneration()
- Cistern.generatePump()

Expected Results:

- When the pump generation condition is satisfied, a new pump is generated at the cistern.
- The pump remains available until picked up by a plumber.

TEST 15: Randomly Break Pump

Sequence:

- System.updateRoundEvents()
- PipeNetwork.getWorkingPumps()
- Pump.breakDown()
- WaterFlowManager.recalculateFlow()

Expected Results:

- If the breakdown condition is satisfied, a working pump becomes broken.
- The new pump state is displayed and water flow is recalculated.

TEST 16: Water Leakage / Flow Logic

Sequence:

- WaterFlowManager.recalculateFlow()
- WaterFlowManager.processSpring(spring)
- WaterFlowManager.registerDeliveredWater(amount)
- WaterFlowManager.registerLeakedWater(amount)
- ScoreBoard.addDeliveredWater(amount)
- ScoreBoard.addLeakedWater(amount)

Expected Results:

- Water moves through the network.
- Water reaching a cistern is counted as delivered.
- Water reaching a punctured pipe or free end is counted as leaked.
- The scoreboard is updated accordingly.

TEST 17: End Game / Declare Winner

Sequence:

- System.checkEndCondition()
- GameTimer.hasExpired()
- System.endGame()
- GameTimer.stop()
- ScoreBoard.determineWinner()

Expected Results:

- When the timer-based end condition is reached, the session stops.

- Delivered and leaked water are compared.
- The winning team is determined and the final result is displayed.

6.2 Evaluation

Name of the team member	Participation (%)
Muhammad Ibrahim Shoeb	16.6%
Arda Gecegörür	16.6 %
Ilgın Tunç	16.6%
Muhammad Hameez Khan	16.6%
Yahya Akhrikhar	16.6%
Aasif Mohd	16.6%

6.3 Tools Used

Tool	**Purpose**	**Description**
ChatGPT (AI)	Writing assistance	Used for proofreading, grammar correction, brainstorming ideas, and clarifying edge cases in the documentation.
Visual Studio Code	Development environment	Used to write, compile, debug, and organize the Java skeleton program files during development.
Draw.io	UML diagram creation	Used by some team members to create communication diagrams.
PlantUML	UML diagram creation	Used by some team members to generate UML diagrams in text-based format.
Gaphor	UML diagram creation	Used to create UML diagrams such as class, sequence, and activity diagrams.
Microsoft Word	Documentation	Used to write, format, and organize the skeleton planning document.
Eclipse 2019-09	Development environment	Used to write, compile, and run the Java skeleton program on the university virtual machine.
Discord	Communication	Used for online team meetings and coordination between team members.

WhatsApp	Communication	Used for informal coordination and quick updates between team members during working sessions.

6.4 Protocol

Start (date & time)	Duration (hours)	Performer(s) name	Activity description
24.03.2026 18:00	2 hours	Ibrahim, Arda, Ilgın, Aasif	Meeting (Discord): The team reviewed the professor's feedback on the submitted Skeleton Planning document. The feedback was analyzed carefully and the main improvement points were identified, including renaming use cases to be role-specific, numbering all use cases, adding a welcome text and command menu to the UI dialogue section, fixing actor naming inside use case descriptions, and verifying edge case handling. Tasks for the next working session were delegated among members: Ibrahim was assigned responsibility for section 6.1.1 (List of Files) and 6.4 (Protocol); Aasif was assigned section 6.1.2 (Compilation); Ilgın was assigned section 6.1.3 (Run); section 6.2 (Evaluation) was agreed to be completed together based on team votes; section 6.0 (Corrections) was agreed to be completed together at the next physical meeting. Arda, Hameez, and Yahya were assigned responsibility for coding, consistency checking, compliance, and formatting. The team agreed that all tasks would be finalized at the next in-person working session on 28.03.2026.
24.03.2026 14:00	7 hours	Ibrahim	Individual work session at BME Library. Ibrahim completed the tasks assigned to him during the Discord meeting, including section 6.1.1 (List of Files), the Tools Used table, and section 6.4 (Protocol), with the exception of the final protocol entry which remains open until the week's work is complete.
28.03.2026 09:30	10 hours	Arda, Ilgın,	Meeting (Corvinus University Library, physical): The full team met in person to finalize the skeleton program submission. The

		Hameez, Yahya, Aasif	session focused on completing section 6.0 (Corrections) together, finalizing all individually assigned sections, performing consistency checks across the document and the runnable program, and preparing the final submission package including the corrected skeleton planning document and the final Analysis Model. Decisions made during the session were recorded and responsibilities were fulfilled according to the delegation agreed on 24.03.2026.